

Handle the completion of the variable photo sequence capture

The **Stopped** event is raised when all of the frames in the sequence have been captured. Update the recording state of your app and update your UI to allow the user to initiate new captures. At this point, you can pass the captured images and frame control values to your image processing code.

C#

```
void OnStopped(object s, object e)
{
    _isRecording = false;
    MyPostProcessingFunction(_images, _frameControlValues, 3);
}
```

Update frame controllers

If you want to perform another variable photo sequence capture with different per frame settings, you don't need to completely reinitialize the

VariablePhotoSequenceCapture. You can either clear the **DesiredFrameControllers** collection and add new frame controllers or you can modify the existing frame controller values. The following example checks the **FrameFlashCapabilities** object to verify that the current device supports flash and flash power for variable photo sequence frames. If so, each frame is updated to enable the flash at 100% power. The exposure compensation values that were previously set for each frame are still active.

C#

```
var varPhotoSeqController =
_mediaCapture.VideoDeviceController.VariablePhotoSequenceController;

if (varPhotoSeqController.FrameCapabilities.Flash.Supported &&
    varPhotoSeqController.FrameCapabilities.Flash.PowerSupported)
{
    for (int i = 0; i < varPhotoSeqController.DesiredFrameControllers.Count;
        i++)
    {
        varPhotoSeqController.DesiredFrameControllers[i].FlashControl.Mode =
        FrameFlashMode.Enable;

        varPhotoSeqController.DesiredFrameControllers[i].FlashControl.PowerPercent =
        100;
    }
}
```

Clean up the variable photo sequence capture

When you are done capturing variable photo sequences or your app is suspending, clean up the variable photo sequence object by calling [FinishAsync](#). Unregister the object's event handlers and set it to null.

C#

```
await _photoSequenceCapture.FinishAsync();
_photoSequenceCapture.PhotoCaptured -= OnPhotoCaptured;
_photoSequenceCapture.Stopped -= OnStopped;
_photoSequenceCapture = null;
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Process media frames with MediaFrameReader

Article • 10/20/2022

This article shows you how to use a [MediaFrameReader](#) with [MediaCapture](#) to get media frames from one or more available sources, including color, depth, and infrared cameras, audio devices, or even custom frame sources such as those that produce skeletal tracking frames. This feature is designed to be used by apps that perform real-time processing of media frames, such as augmented reality and depth-aware camera apps.

If you are interested in simply capturing video or photos, such as a typical photography app, then you probably want to use one of the other capture techniques supported by [MediaCapture](#). For a list of available media capture techniques and articles showing how to use them, see [Camera](#).

ⓘ Note

The features discussed in this article are only available starting with Windows 10, version 1607.

ⓘ Note

There is a Universal Windows app sample that demonstrates using **MediaFrameReader** to display frames from different frame sources, including color, depth, and infrared cameras. For more information, see [Camera frames sample](#) .

ⓘ Note

A new set of APIs for using **MediaFrameReader** with audio data were introduced in Windows 10, version 1803. For more information, see [Process audio frames with MediaFrameReader](#).

Setting up your project

As with any app that uses **MediaCapture**, you must declare that your app uses the *webcam* capability before attempting to access any camera device. If your app will

capture from an audio device, you should also declare the *microphone* device capability.

Add capabilities to the app manifest

1. In Microsoft Visual Studio, in **Solution Explorer**, open the designer for the application manifest by double-clicking the **package.appxmanifest** item.
2. Select the **Capabilities** tab.
3. Check the box for **Webcam** and the box for **Microphone**.
4. For access to the Pictures and Videos library check the boxes for **Pictures Library** and the box for **Videos Library**.

The example code in this article uses APIs from the following namespaces, in addition to those included by the default project template.

C#

```
using Windows.Media.Capture.Frames;
using Windows.Devices.Enumeration;
using Windows.Media.Capture;
using Windows.UI.Xaml.Media.Imaging;
using Windows.Media.MediaProperties;
using Windows.Graphics.Imaging;
using System.Threading;
using Windows.UI.Core;
using System.Threading.Tasks;
using Windows.Media.Core;
using System.Diagnostics;
using Windows.Media;
using Windows.Media.Devices;
using Windows.Media.Audio;
```

Select frame sources and frame source groups

Many apps that process media frames need to get frames from multiple sources at once, such as a device's color and depth cameras. The [MediaFrameSourceGroup](#) object represents a set of media frame sources that can be used simultaneously. Call the static method [MediaFrameSourceGroup.FindAllAsync](#) to get a list of all of the groups of frame sources supported by the current device.

C#

```
var frameSourceGroups = await MediaFrameSourceGroup.FindAllAsync();
```

You can also create a [DeviceWatcher](#) using [DeviceInformation.CreateWatcher](#) and the value returned from [MediaFrameSourceGroup.GetDeviceSelector](#) to receive

notifications when the available frame source groups on the device changes, such as when an external camera is plugged in. For more information see [Enumerate devices](#).

A [MediaFrameSourceGroup](#) has a collection of [MediaFrameSourceInfo](#) objects that describe the frame sources included in the group. After retrieving the frame source groups available on the device, you can select the group that exposes the frame sources you are interested in.

The following example shows the simplest way to select a frame source group. This code simply loops over all of the available groups and then loops over each item in the [SourceInfos](#) collection. Each [MediaFrameSourceInfo](#) is checked to see if it supports the features we are seeking. In this case, the [MediaStreamType](#) property is checked for the value [VideoPreview](#), meaning the device provides a video preview stream, and the [SourceKind](#) property is checked for the value [Color](#), indicating that the source provides color frames.

C#

```
var frameSourceGroups = await MediaFrameSourceGroup.FindAllAsync();

MediaFrameSourceGroup selectedGroup = null;
MediaFrameSourceInfo colorSourceInfo = null;

foreach (var sourceGroup in frameSourceGroups)
{
    foreach (var sourceInfo in sourceGroup.SourceInfos)
    {
        if (sourceInfo.MediaStreamType == MediaStreamType.VideoPreview
            && sourceInfo.SourceKind == MediaFrameSourceKind.Color)
        {
            colorSourceInfo = sourceInfo;
            break;
        }
    }
    if (colorSourceInfo != null)
    {
        selectedGroup = sourceGroup;
        break;
    }
}
```

This method of identifying the desired frame source group and frame sources works for simple cases, but if you want to select frame sources based on more complex criteria, it can quickly become cumbersome. Another method is to use Linq syntax and anonymous objects to make the selection. The following example uses the [Select](#) extension method to transform the [MediaFrameSourceGroup](#) objects in the *frameSourceGroups* list into an anonymous object with two fields: *sourceGroup*,

representing the group itself, and *colorSourceInfo*, which represents the color frame source in the group. The *colorSourceInfo* field is set to the result of **FirstOrDefault**, which selects the first object for which the provided predicate resolves to true. In this case, the predicate is true if the stream type is **VideoPreview**, the source kind is **Color**, and if the camera is on the front panel of the device.

From the list of anonymous objects returned from the query described above, the **Where** extension method is used to select only those objects where the *colorSourceInfo* field is not null. Finally, **FirstOrDefault** is called to select the first item in the list.

Now you can use the fields of the selected object to get references to the selected **MediaFrameSourceGroup** and the **MediaFrameSourceInfo** object representing the color camera. These will be used later to initialize the **MediaCapture** object and create a **MediaFrameReader** for the selected source. Finally, you should test to see if the source group is null, meaning the current device doesn't have your requested capture sources.

C#

```
var selectedGroupObjects = frameSourceGroups.Select(group =>
    new
    {
        sourceGroup = group,
        colorSourceInfo = group.SourceInfos.FirstOrDefault((sourceInfo) =>
        {
            // On Xbox/Kinect, omit the MediaStreamType and EnclosureLocation
            tests
            return sourceInfo.MediaStreamType == MediaStreamType.VideoPreview
                && sourceInfo.SourceKind == MediaFrameSourceKind.Color
                && sourceInfo.DeviceInformation?.EnclosureLocation.Panel ==
Windows.Devices.Enumeration.Panel.Front;
        })

    }).Where(t => t.colorSourceInfo != null)
    .FirstOrDefault();

MediaFrameSourceGroup selectedGroup = selectedGroupObjects?.sourceGroup;
MediaFrameSourceInfo colorSourceInfo =
selectedGroupObjects?.colorSourceInfo;

if (selectedGroup == null)
{
    return;
}
```

The following example uses a similar technique as described above to select a source group that contains color, depth, and infrared cameras.

C#

```

var allGroups = await MediaFrameSourceGroup.FindAllAsync();
var eligibleGroups = allGroups.Select(g => new
{
    Group = g,

    // For each source kind, find the source which offers that kind of media
    frame,
    // or null if there is no such source.
    SourceInfos = new MediaFrameSourceInfo[]
    {
        g.SourceInfos.FirstOrDefault(info => info.SourceKind ==
MediaFrameSourceKind.Color),
        g.SourceInfos.FirstOrDefault(info => info.SourceKind ==
MediaFrameSourceKind.Depth),
        g.SourceInfos.FirstOrDefault(info => info.SourceKind ==
MediaFrameSourceKind.Infrared),
    }
}).Where(g => g.SourceInfos.Any(info => info != null)).ToList();

if (eligibleGroups.Count == 0)
{
    System.Diagnostics.Debug.WriteLine("No source group with color, depth or
infrared found.");
    return;
}

var selectedGroupIndex = 0; // Select the first eligible group
MediaFrameSourceGroup selectedGroup =
eligibleGroups[selectedGroupIndex].Group;
MediaFrameSourceInfo colorSourceInfo =
eligibleGroups[selectedGroupIndex].SourceInfos[0];
MediaFrameSourceInfo infraredSourceInfo =
eligibleGroups[selectedGroupIndex].SourceInfos[1];
MediaFrameSourceInfo depthSourceInfo =
eligibleGroups[selectedGroupIndex].SourceInfos[2];

```

ⓘ Note

Starting with Windows 10, version 1803, you can use the [MediaCaptureVideoProfile](#) class to select a media frame source with a set of desired capabilities. For more information, see the section **Use video profiles to select a frame source** later in this article.

Initialize the MediaCapture object to use the selected frame source group

The next step is to initialize the **MediaCapture** object to use the frame source group you selected in the previous step.

The **MediaCapture** object is typically used from multiple locations within your app, so you should declare a class member variable to hold it.

C#

```
MediaCapture mediaCapture;
```

Create an instance of the **MediaCapture** object by calling the constructor. Next, create a **MediaCaptureInitializationSettings** object that will be used to initialize the **MediaCapture** object. In this example, the following settings are used:

- **SourceGroup** - This tells the system which source group you will be using to get frames. Remember that the source group defines a set of media frame sources that can be used simultaneously.
- **SharingMode** - This tells the system whether you need exclusive control over the capture source devices. If you set this to **ExclusiveControl**, it means that you can change the settings of the capture device, such as the format of the frames it produces, but this means that if another app already has exclusive control, your app will fail when it tries to initialize the media capture device. If you set this to **SharedReadOnly**, you can receive frames from the frame sources even if they are in use by another app, but you can't change the settings for the devices.
- **MemoryPreference** - If you specify **CPU**, the system will use CPU memory which guarantees that when frames arrive, they will be available as **SoftwareBitmap** objects. If you specify **Auto**, the system will dynamically choose the optimal memory location to store frames. If the system chooses to use GPU memory, the media frames will arrive as an **IDirect3DSurface** object and not as a **SoftwareBitmap**.
- **StreamingCaptureMode** - Set this to **Video** to indicate that audio doesn't need to be streamed.

Call **InitializeAsync** to initialize the **MediaCapture** with your desired settings. Be sure to call this within a *try* block in case initialization fails.

C#

```
mediaCapture = new MediaCapture();

var settings = new MediaCaptureInitializationSettings()
{
    SourceGroup = selectedGroup,
    SharingMode = MediaCaptureSharingMode.ExclusiveControl,
```

```

        MemoryPreference = MediaCaptureMemoryPreference.Cpu,
        StreamingCaptureMode = StreamingCaptureMode.Video
    };
    try
    {
        await mediaCapture.InitializeAsync(settings);
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine("MediaCapture initialization failed:
" + ex.Message);
        return;
    }
}

```

Set the preferred format for the frame source

To set the preferred format for a frame source, you need to get a [MediaFrameSource](#) object representing the source. You get this object by accessing the [Frames](#) dictionary of the initialized [MediaCapture](#) object, specifying the identifier of the frame source you want to use. This is why we saved the [MediaFrameSourceInfo](#) object when we were selecting a frame source group.

The [MediaFrameSource.SupportedFormats](#) property contains a list of [MediaFrameFormat](#) objects describing the supported formats for the frame source. Use the **Where** Linq extension method to select a format based on desired properties. In this example, a format is selected that has a width of 1080 pixels and can supply frames in 32-bit RGB format. The **FirstOrDefault** extension method selects the first entry in the list. If the selected format is null, then the requested format is not supported by the frame source. If the format is supported, you can request that the source use this format by calling [SetFormatAsync](#).

C#

```

var colorFrameSource = mediaCapture.FrameSources[colorSourceInfo.Id];
var preferredFormat = colorFrameSource.SupportedFormats.Where(format =>
{
    return format.VideoFormat.Width >= 1080
    && format.Subtype == MediaEncodingSubtypes.Argb32;
}).FirstOrDefault();

if (preferredFormat == null)
{
    // Our desired format is not supported
    return;
}

```

```
await colorFrameSource.SetFormatAsync(preferredFormat);
```

Create a frame reader for the frame source

To receive frames for a media frame source, use a [MediaFrameReader](#).

C#

```
MediaFrameReader mediaFrameReader;
```

Instantiate the frame reader by calling [CreateFrameReaderAsync](#) on your initialized **MediaCapture** object. The first argument to this method is the frame source from which you want to receive frames. You can create a separate frame reader for each frame source you want to use. The second argument tells the system the output format in which you want frames to arrive. This can save you from having to do your own conversions to frames as they arrive. Note that if you specify a format that is not supported by the frame source, an exception will be thrown, so be sure that this value is in the [SupportedFormats](#) collection.

After creating the frame reader, register a handler for the [FrameArrived](#) event which is raised whenever a new frame is available from the source.

Tell the system to start reading frames from the source by calling [StartAsync](#).

C#

```
mediaFrameReader = await  
mediaCapture.CreateFrameReaderAsync(colorFrameSource,  
MediaEncodingSubtypes.Argb32);  
mediaFrameReader.FrameArrived += ColorFrameReader_FrameArrived;  
await mediaFrameReader.StartAsync();
```

Handle the frame arrived event

The [MediaFrameReader.FrameArrived](#) event is raised whenever a new frame is available. You can choose to process every frame that arrives or only use frames when you need them. Because the frame reader raises the event on its own thread, you may need to implement some synchronization logic to make sure that you aren't attempting to access the same data from multiple threads. This section shows you how to synchronize drawing color frames to an image control in a XAML page. This scenario

addresses the additional synchronization constraint that requires all updates to XAML controls be performed on the UI thread.

The first step in displaying frames in XAML is to create an Image control.

XML

```
<Image x:Name="imageElement" Width="320" Height="240" />
```

In your code behind page, declare a class member variable of type **SoftwareBitmap** which will be used as a back buffer that all incoming images will be copied to. Note that the image data itself isn't copied, just the object references. Also, declare a boolean to track whether our UI operation is currently running.

C#

```
private SoftwareBitmap backBuffer;  
private bool taskRunning = false;
```

Because the frames will arrive as **SoftwareBitmap** objects, you need to create a **SoftwareBitmapSource** object which allows you to use a **SoftwareBitmap** as the source for a XAML **Control**. You should set the image source somewhere in your code before you start the frame reader.

C#

```
imageElement.Source = new SoftwareBitmapSource();
```

Now it's time to implement the **FrameArrived** event handler. When the handler is called, the *sender* parameter contains a reference to the **MediaFrameReader** object which raised the event. Call **TryAcquireLatestFrame** on this object to attempt to get the latest frame. As the name implies, **TryAcquireLatestFrame** may not succeed in returning a frame. So, when you access the **VideoMediaFrame** and then **SoftwareBitmap** properties, be sure to test for null. In this example the null conditional operator `?` is used to access the **SoftwareBitmap** and then the retrieved object is checked for null.

The **Image** control can only display images in BRGA8 format with either pre-multiplied or no alpha. If the arriving frame is not in that format, the static method **Convert** is used to convert the software bitmap to the correct format.

Next, the **Interlocked.Exchange** method is used to swap the reference of to arriving bitmap with the backbuffer bitmap. This method swaps these references in an atomic

operation that is thread-safe. After swapping, the old backbuffer image, now in the *softwareBitmap* variable is disposed of to clean up its resources.

Next, the **CoreDispatcher** associated with the **Image** element is used to create a task that will run on the UI thread by calling **RunAsync**. Because the asynchronous tasks will be performed within the task, the lambda expression passed to **RunAsync** is declared with the *async* keyword.

Within the task, the *_taskRunning* variable is checked to make sure that only one instance of the task is running at a time. If the task isn't already running, *_taskRunning* is set to true to prevent the task from running again. In a *while* loop, **Interlocked.Exchange** is called to copy from the backbuffer into a temporary **SoftwareBitmap** until the backbuffer image is null. For each time the temporary bitmap is populated, the **Source** property of the **Image** is cast to a **SoftwareBitmapSource**, and then **SetBitmapAsync** is called to set the source of the image.

Finally, the *_taskRunning* variable is set back to false so that the task can be run again the next time the handler is called.

ⓘ Note

If you access the **SoftwareBitmap** or **Direct3DSurface** objects provided by the **VideoMediaFrame** property of a **MediaFrameReference**, the system creates a strong reference to these objects, which means that they will not be disposed when you call **Dispose** on the containing **MediaFrameReference**. You must explicitly call the **Dispose** method of the **SoftwareBitmap** or **Direct3DSurface** directly for the objects to be immediately disposed. Otherwise, the garbage collector will eventually free the memory for these objects, but you can't know when this will occur, and if the number of allocated bitmaps or surfaces exceeds the maximum amount allowed by the system, the flow of new frames will stop. You can copy retrieved frames, using the **SoftwareBitmap.Copy** method for example, and then release the original frames to overcome this limitation. Also, if you create the **MediaFrameReader** using the overload **CreateFrameReaderAsync(Windows.Media.Capture.Frames.MediaFrameSource inputSource, System.String outputSubtype, Windows.Graphics.Imaging.BitmapSize outputSize)** or **CreateFrameReaderAsync(Windows.Media.Capture.Frames.MediaFrameSource inputSource, System.String outputSubtype)**, the frames returned are copies of the original frame data and so they do not cause frame acquisition to halt when they are retained.

```

private void ColorFrameReader_FrameArrived(MediaFrameReader sender,
MediaFrameArrivedEventArgs args)
{
    var mediaFrameReference = sender.TryAcquireLatestFrame();
    var videoMediaFrame = mediaFrameReference?.VideoMediaFrame;
    var softwareBitmap = videoMediaFrame?.SoftwareBitmap;

    if (softwareBitmap != null)
    {
        if (softwareBitmap.BitmapPixelFormat !=
Windows.Graphics.Imaging.BitmapPixelFormat.Bgra8 ||
            softwareBitmap.BitmapAlphaMode !=
Windows.Graphics.Imaging.BitmapAlphaMode.Premultiplied)
        {
            softwareBitmap = SoftwareBitmap.Convert(softwareBitmap,
BitmapPixelFormat.Bgra8, BitmapAlphaMode.Premultiplied);
        }

        // Swap the processed frame to _backBuffer and dispose of the unused
image.
        softwareBitmap = Interlocked.Exchange(ref backBuffer,
softwareBitmap);
        softwareBitmap?.Dispose();

        // Changes to XAML ImageElement must happen on UI thread through
Dispatcher
        var task =
imageElement.Dispatcher.RunAsync(CoreDispatcherPriority.Normal,
            async () =>
            {
                // Don't let two copies of this task run at the same time.
                if (taskRunning)
                {
                    return;
                }
                taskRunning = true;

                // Keep draining frames from the backbuffer until the
backbuffer is empty.
                SoftwareBitmap latestBitmap;
                while ((latestBitmap = Interlocked.Exchange(ref backBuffer,
null)) != null)
                {
                    var imageSource =
(SoftwareBitmapSource)imageElement.Source;
                    await imageSource.SetBitmapAsync(latestBitmap);
                    latestBitmap.Dispose();
                }

                taskRunning = false;
            });
    }
}

```

```
mediaFrameReference.Dispose();  
}
```

Cleanup resources

When you are done reading frames, be sure to stop the media frame reader by calling [StopAsync](#), unregistering the **FrameArrived** handler, and disposing of the **MediaCapture** object.

C#

```
await mediaFrameReader.StopAsync();  
mediaFrameReader.FrameArrived -= ColorFrameReader_FrameArrived;  
mediaCapture.Dispose();  
mediaCapture = null;
```

For more information about cleaning up media capture objects when your application is suspended, see [Display the camera preview](#).

The FrameRenderer helper class

The Universal Windows [Camera frames sample](#) [↗](#) provides a helper class that makes it easy to display the frames from color, infrared, and depth sources in your app. Typically, you will want to do something more with depth and infrared data than just display it to the screen, but this helper class is a helpful tool for demonstrating the frame reader feature and for debugging your own frame reader implementation.

The **FrameRenderer** helper class implements the following methods.

- **FrameRenderer** constructor - The constructor initializes the helper class to use the XAML **Image** element you pass in for displaying media frames.
- **ProcessFrame** - This method displays a media frame, represented by a **MediaFrameReference**, in the **Image** element you passed into the constructor. You should typically call this method from your **FrameArrived** event handler, passing in the frame returned by **TryAcquireLatestFrame**.
- **ConvertToDisplayableImage** - This methods checks the format of the media frame and, if necessary, converts it to a displayable format. For color images, this means making sure that the color format is BGRA8 and that the bitmap alpha mode is premultiplied. For depth or infrared frames, each scanline is processed to convert the depth or infrared values to a psuedocolor gradient, using the **PsuedoColorHelper** class that is also included in the sample and listed below.

ⓘ Note

In order to do pixel manipulation on **SoftwareBitmap** images, you must access a native memory buffer. To do this, you must use the `IMemoryBufferByteAccess` COM interface included in the code listing below and you must update your project properties to allow compilation of unsafe code. For more information, see [Create, edit, and save bitmap images](#).

C#

```
[ComImport]
[Guid("5B0D3235-4DBA-4D44-865E-8F1D0E4FD04D")]
[InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
unsafe interface IMemoryBufferByteAccess
{
    void GetBuffer(out byte* buffer, out uint capacity);
}
```

C#

```
class FrameRenderer
{
    private Image _imageElement;
    private SoftwareBitmap _backBuffer;
    private bool _taskRunning = false;

    public FrameRenderer(Image imageElement)
    {
        _imageElement = imageElement;
        _imageElement.Source = new SoftwareBitmapSource();
    }

    // Processes a MediaFrameReference and displays it in a XAML image
    control
    public void ProcessFrame(MediaFrameReference frame)
    {
        var softwareBitmap =
            FrameRenderer.ConvertToDisplayableImage(frame?.VideoMediaFrame);
        if (softwareBitmap != null)
        {
            // Swap the processed frame to _backBuffer and trigger UI thread
            to render it
            softwareBitmap = Interlocked.Exchange(ref _backBuffer,
                softwareBitmap);

            // UI thread always reset _backBuffer before using it. Unused
            bitmap should be disposed.
            softwareBitmap?.Dispose();
        }
    }
}
```

```

        // Changes to xaml ImageElement must happen in UI thread through
Dispatcher
        var task =
_imageElement.Dispatcher.RunAsync(CoreDispatcherPriority.Normal,
        async () =>
        {
            // Don't let two copies of this task run at the same
time.

            if (_taskRunning)
            {
                return;
            }
            _taskRunning = true;

            // Keep draining frames from the backbuffer until the
backbuffer is empty.
            SoftwareBitmap latestBitmap;
            while ((latestBitmap = Interlocked.Exchange(ref
_backBuffer, null)) != null)
            {
                var imageSource =
(SoftwareBitmapSource)_imageElement.Source;
                await imageSource.SetBitmapAsync(latestBitmap);
                latestBitmap.Dispose();
            }

            _taskRunning = false;
        });
    }
}

```

// Function delegate that transforms a scanline from an input image to an output image.

```

private unsafe delegate void TransformScanline(int pixelWidth, byte*
inputRowBytes, byte* outputRowBytes);

```

```

/// <summary>
/// Determines the subtype to request from the MediaFrameReader that
will result in
/// a frame that can be rendered by ConvertToDisplayableImage.
/// </summary>
/// <returns>Subtype string to request, or null if subtype is not
renderable.</returns>

```

```

public static string GetSubtypeForFrameReader(MediaFrameSourceKind kind,
MediaFrameFormat format)

```

```

{
    // Note that media encoding subtypes may differ in case.
    // https://docs.microsoft.com/en-
us/uwp/api/Windows.Media.MediaProperties.MediaEncodingSubtypes

```

```

    string subtype = format.Subtype;
    switch (kind)

```

```

    {
        // For color sources, we accept anything and request that it be
        converted to Bgra8.
        case MediaFrameSourceKind.Color:
            return
Windows.Media.MediaProperties.MediaEncodingSubtypes.Bgra8;

        // The only depth format we can render is D16.
        case MediaFrameSourceKind.Depth:
            return String.Equals(subtype,
Windows.Media.MediaProperties.MediaEncodingSubtypes.D16,
StringComparison.OrdinalIgnoreCase) ? subtype : null;

        // The only infrared formats we can render are L8 and L16.
        case MediaFrameSourceKind.Infrared:
            return (String.Equals(subtype,
Windows.Media.MediaProperties.MediaEncodingSubtypes.L8,
StringComparison.OrdinalIgnoreCase) ||
                String.Equals(subtype,
Windows.Media.MediaProperties.MediaEncodingSubtypes.L16,
StringComparison.OrdinalIgnoreCase)) ? subtype : null;

        // No other source kinds are supported by this class.
        default:
            return null;
    }
}

/// <summary>
/// Converts a frame to a SoftwareBitmap of a valid format to display in
an Image control.
/// </summary>
/// <param name="inputFrame">Frame to convert.</param>

public static unsafe SoftwareBitmap
ConvertToDisplayableImage(VideoMediaFrame inputFrame)
{
    SoftwareBitmap result = null;
    using (var inputBitmap = inputFrame?.SoftwareBitmap)
    {
        if (inputBitmap != null)
        {
            switch (inputFrame.FrameReference.SourceKind)
            {
                case MediaFrameSourceKind.Color:
                    // XAML requires Bgra8 with premultiplied alpha.
                    // We requested Bgra8 from the MediaFrameReader, so
all that's
                    // left is fixing the alpha channel if necessary.
                    if (inputBitmap.BitmapPixelFormat !=
BitmapPixelFormat.Bgra8)
                    {
                        System.Diagnostics.Debug.WriteLine("Color frame
in unexpected format.");
                    }
                }
            }
        }
    }
}

```

```

        else if (inputBitmap.BitmapAlphaMode ==
BitmapAlphaMode.Premultiplied)
        {
            // Already in the correct format.
            result = SoftwareBitmap.Copy(inputBitmap);
        }
        else
        {
            // Convert to premultiplied alpha.
            result = SoftwareBitmap.Convert(inputBitmap,
BitmapPixelFormat.Bgra8, BitmapAlphaMode.Premultiplied);
        }
        break;

    case MediaFrameSourceKind.Depth:
        // We requested D16 from the MediaFrameReader, so
the frame should
        // be in Gray16 format.
        if (inputBitmap.BitmapPixelFormat ==
BitmapPixelFormat.Gray16)
        {
            // Use a special pseudo color to render 16 bits
depth frame.

            var depthScale =
(float)inputFrame.DepthMediaFrame.DepthFormat.DepthScaleInMeters;
            var minReliableDepth =
inputFrame.DepthMediaFrame.MinReliableDepth;
            var maxReliableDepth =
inputFrame.DepthMediaFrame.MaxReliableDepth;
            result = TransformBitmap(inputBitmap, (w, i, o)
=> PseudoColorHelper.PseudoColorForDepth(w, i, o, depthScale,
minReliableDepth, maxReliableDepth));
        }
        else
        {
            System.Diagnostics.Debug.WriteLine("Depth frame
in unexpected format.");
        }
        break;

    case MediaFrameSourceKind.Infrared:
        // We requested L8 or L16 from the MediaFrameReader,
so the frame should
        // be in Gray8 or Gray16 format.
        switch (inputBitmap.BitmapPixelFormat)
        {
            case BitmapPixelFormat.Gray16:
                // Use pseudo color to render 16 bits
frames.

                result = TransformBitmap(inputBitmap,
PseudoColorHelper.PseudoColorFor16BitInfrared);
                break;

            case BitmapPixelFormat.Gray8:
                // Use pseudo color to render 8 bits frames.

```

```

        result = TransformBitmap(inputBitmap,
PseudoColorHelper.PseudoColorFor8BitInfrared);
        break;
        default:
            System.Diagnostics.Debug.WriteLine("Infrared
frame in unexpected format.");
            break;
    }
    break;
}
}
}

return result;
}

```

```

/// <summary>
/// Transform image into Bgra8 image using given transform method.
/// </summary>
/// <param name="softwareBitmap">Input image to transform.</param>
/// <param name="transformScanline">Method to map pixels in a scanline.
</param>

```

```

private static unsafe SoftwareBitmap TransformBitmap(SoftwareBitmap
softwareBitmap, TransformScanline transformScanline)
{
    // XAML Image control only supports premultiplied Bgra8 format.
    var outputBitmap = new SoftwareBitmap(BitmapPixelFormat.Bgra8,
        softwareBitmap.PixelWidth, softwareBitmap.PixelHeight,
        BitmapAlphaMode.Premultiplied);

    using (var input =
softwareBitmap.LockBuffer(BitmapBufferAccessMode.Read))
        using (var output =
outputBitmap.LockBuffer(BitmapBufferAccessMode.Write))
        {
            // Get stride values to calculate buffer position for a given
            pixel x and y position.
            int inputStride = input.GetPlaneDescription(0).Stride;
            int outputStride = output.GetPlaneDescription(0).Stride;
            int pixelWidth = softwareBitmap.PixelWidth;
            int pixelHeight = softwareBitmap.PixelHeight;

            using (var outputReference = output.CreateReference())
            using (var inputReference = input.CreateReference())
            {
                // Get input and output byte access buffers.
                byte* inputBytes;
                uint inputCapacity;
                ((IMemoryBufferByteAccess)inputReference).GetBuffer(out
inputBytes, out inputCapacity);
                byte* outputBytes;
                uint outputCapacity;
            }
        }
    }
}

```

```

        ((IMemoryBufferByteAccess)outputReference).GetBuffer(out
outputBytes, out outputCapacity);

        // Iterate over all pixels and store converted value.
        for (int y = 0; y < pixelHeight; y++)
        {
            byte* inputRowBytes = inputBytes + y * inputStride;
            byte* outputRowBytes = outputBytes + y * outputStride;

            transformScanline(pixelWidth, inputRowBytes,
outputRowBytes);
        }
    }

    return outputBitmap;
}

```

```

/// <summary>
/// A helper class to manage look-up-table for pseudo-colors.
/// </summary>

```

```

private static class PseudoColorHelper
{
    #region Constructor, private members and methods

    private const int TableSize = 1024;    // Look up table size
    private static readonly uint[] PseudoColorTable;
    private static readonly uint[] InfraredRampTable;

    // Color palette mapping value from 0 to 1 to blue to red colors.
    private static readonly Color[] ColorRamp =
    {
        Color.FromArgb(a:0xFF, r:0x7F, g:0x00, b:0x00),
        Color.FromArgb(a:0xFF, r:0xFF, g:0x00, b:0x00),
        Color.FromArgb(a:0xFF, r:0xFF, g:0x7F, b:0x00),
        Color.FromArgb(a:0xFF, r:0xFF, g:0xFF, b:0x00),
        Color.FromArgb(a:0xFF, r:0x7F, g:0xFF, b:0x7F),
        Color.FromArgb(a:0xFF, r:0x00, g:0xFF, b:0xFF),
        Color.FromArgb(a:0xFF, r:0x00, g:0x7F, b:0xFF),
        Color.FromArgb(a:0xFF, r:0x00, g:0x00, b:0xFF),
        Color.FromArgb(a:0xFF, r:0x00, g:0x00, b:0x7F),
    };

    static PseudoColorHelper()
    {
        PseudoColorTable = InitializePseudoColorLut();
        InfraredRampTable = InitializeInfraredRampLut();
    }

    /// <summary>
    /// Maps an input infrared value between [0, 1] to corrected value
    between [0, 1].

```

```

    /// </summary>
    /// <param name="value">Input value between [0, 1].</param>
    [MethodImpl(MethodImplOptions.AggressiveInlining)] // Tell the
    compiler to inline this method to improve performance

    private static uint InfraredColor(float value)
    {
        int index = (int)(value * TableSize);
        index = index < 0 ? 0 : index > TableSize - 1 ? TableSize - 1 :
index;

        return InfraredRampTable[index];
    }

    /// <summary>
    /// Initializes the pseudo-color look up table for infrared pixels
    /// </summary>

    private static uint[] InitializeInfraredRampLut()
    {
        uint[] lut = new uint[TableSize];
        for (int i = 0; i < TableSize; i++)
        {
            var value = (float)i / TableSize;
            // Adjust to increase color change between lower values in
infrared images

            var alpha = (float)Math.Pow(1 - value, 12);
            lut[i] = ColorRampInterpolation(alpha);
        }

        return lut;
    }

    /// <summary>
    /// Initializes pseudo-color look up table for depth pixels
    /// </summary>
    private static uint[] InitializePseudoColorLut()
    {
        uint[] lut = new uint[TableSize];
        for (int i = 0; i < TableSize; i++)
        {
            lut[i] = ColorRampInterpolation((float)i / TableSize);
        }

        return lut;
    }

    /// <summary>
    /// Maps a float value to a pseudo-color pixel
    /// </summary>
    private static uint ColorRampInterpolation(float value)

```

```

{
    // Map value to surrounding indexes on the color ramp
    int rampSteps = ColorRamp.Length - 1;
    float scaled = value * rampSteps;
    int integer = (int)scaled;
    int index =
        integer < 0 ? 0 :
        integer >= rampSteps - 1 ? rampSteps - 1 :
        integer;

    Color prev = ColorRamp[index];
    Color next = ColorRamp[index + 1];

    // Set color based on ratio of closeness between the surrounding
colors
    uint alpha = (uint)((scaled - integer) * 255);
    uint beta = 255 - alpha;
    return
        ((prev.A * beta + next.A * alpha) / 255) << 24 | // Alpha
        ((prev.R * beta + next.R * alpha) / 255) << 16 | // Red
        ((prev.G * beta + next.G * alpha) / 255) << 8 | // Green
        ((prev.B * beta + next.B * alpha) / 255); // Blue
}

/// <summary>
/// Maps a value in [0, 1] to a pseudo RGBA color.
/// </summary>
/// <param name="value">Input value between [0, 1].</param>
[MethodImpl(MethodImplOptions.AggressiveInlining)]

private static uint PseudoColor(float value)
{
    int index = (int)(value * TableSize);
    index = index < 0 ? 0 : index > TableSize - 1 ? TableSize - 1 :
index;
    return PseudoColorTable[index];
}

#endregion

/// <summary>
/// Maps each pixel in a scanline from a 16 bit depth value to a
pseudo-color pixel.
/// </summary>
/// <param name="pixelWidth">Width of the input scanline, in pixels.
</param>
/// <param name="inputRowBytes">Pointer to the start of the input
scanline.</param>
/// <param name="outputRowBytes">Pointer to the start of the output
scanline.</param>
/// <param name="depthScale">Physical distance that corresponds to
one unit in the input scanline.</param>
/// <param name="minReliableDepth">Shortest distance at which the
sensor can provide reliable measurements.</param>

```

```

    /// <param name="maxReliableDepth">Furthest distance at which the
    sensor can provide reliable measurements.</param>

    public static unsafe void PseudoColorForDepth(int pixelWidth, byte*
inputRowBytes, byte* outputRowBytes, float depthScale, float
minReliableDepth, float maxReliableDepth)
    {
        // Visualize space in front of your desktop.
        float minInMeters = minReliableDepth * depthScale;
        float maxInMeters = maxReliableDepth * depthScale;
        float one_min = 1.0f / minInMeters;
        float range = 1.0f / maxInMeters - one_min;

        ushort* inputRow = (ushort*)inputRowBytes;
        uint* outputRow = (uint*)outputRowBytes;

        for (int x = 0; x < pixelWidth; x++)
        {
            var depth = inputRow[x] * depthScale;

            if (depth == 0)
            {
                // Map invalid depth values to transparent pixels.
                // This happens when depth information cannot be
                calculated, e.g. when objects are too close.
                outputRow[x] = 0;
            }
            else
            {
                var alpha = (1.0f / depth - one_min) / range;
                outputRow[x] = PseudoColor(alpha * alpha);
            }
        }
    }

    /// <summary>
    /// Maps each pixel in a scanline from a 8 bit infrared value to a
    pseudo-color pixel.
    /// </summary>
    /// /// <param name="pixelWidth">Width of the input scanline, in
    pixels.</param>
    /// <param name="inputRowBytes">Pointer to the start of the input
    scanline.</param>
    /// <param name="outputRowBytes">Pointer to the start of the output
    scanline.</param>

    public static unsafe void PseudoColorFor8BitInfrared(
        int pixelWidth, byte* inputRowBytes, byte* outputRowBytes)
    {
        byte* inputRow = inputRowBytes;
        uint* outputRow = (uint*)outputRowBytes;

        for (int x = 0; x < pixelWidth; x++)

```

```

        {
            outputRow[x] = InfraredColor(inputRow[x] /
(float)Byte.MaxValue);
        }
    }

    /// <summary>
    /// Maps each pixel in a scanline from a 16 bit infrared value to a
pseudo-color pixel.
    /// </summary>
    /// <param name="pixelWidth">Width of the input scanline.</param>
    /// <param name="inputRowBytes">Pointer to the start of the input
scanline.</param>
    /// <param name="outputRowBytes">Pointer to the start of the output
scanline.</param>

    public static unsafe void PseudoColorFor16BitInfrared(int
pixelWidth, byte* inputRowBytes, byte* outputRowBytes)
    {
        ushort* inputRow = (ushort*)inputRowBytes;
        uint* outputRow = (uint*)outputRowBytes;

        for (int x = 0; x < pixelWidth; x++)
        {
            outputRow[x] = InfraredColor(inputRow[x] /
(float)UInt16.MaxValue);
        }
    }
}

// Displays the provided softwareBitmap in a XAML image control.
public void PresentSoftwareBitmap(SoftwareBitmap softwareBitmap)
{
    if (softwareBitmap != null)
    {
        // Swap the processed frame to _backBuffer and trigger UI thread
to render it
        softwareBitmap = Interlocked.Exchange(ref _backBuffer,
softwareBitmap);

        // UI thread always reset _backBuffer before using it. Unused
bitmap should be disposed.
        softwareBitmap?.Dispose();

        // Changes to xaml ImageElement must happen in UI thread through
Dispatcher
        var task =
_imageElement.Dispatcher.RunAsync(CoreDispatcherPriority.Normal,
async () =>
        {
            // Don't let two copies of this task run at the same
time.

            if (_taskRunning)
            {

```

```

        return;
    }
    _taskRunning = true;

    // Keep draining frames from the backbuffer until the
    backbuffer is empty.
    SoftwareBitmap latestBitmap;
    while ((latestBitmap = Interlocked.Exchange(ref
_backBuffer, null)) != null)
    {
        var imageSource =
        (SoftwareBitmapSource)_imageElement.Source;
        await imageSource.SetBitmapAsync(latestBitmap);
        latestBitmap.Dispose();
    }

    _taskRunning = false;
});
}
}
}

```

Use MultiSourceMediaFrameReader to get time-correlated frames from multiple sources

Starting with Windows 10, version 1607, you can use [MultiSourceMediaFrameReader](#) to receive time-correlated frames from multiple sources. This API makes it easier to do processing that requires frames from multiple sources that were taken in close temporal proximity, such as using the [DepthCorrelatedCoordinateMapper](#) class. One limitation of using this new method is that frame-arrived events are only raised at the rate of the slowest capture source. Extra frames from faster sources will be dropped. Also, because the system expects frames to arrive from different sources at different rates, it does not automatically recognize if a source has stopped generating frames altogether. The example code in this section shows how to use an event to create your own timeout logic that gets invoked if correlated frames don't arrive within an app-defined time limit.

The steps for using [MultiSourceMediaFrameReader](#) are similar to the steps for using [MediaFrameReader](#) described previously in this article. This example will use a color source and a depth source. Declare some string variables to store the media frame source IDs that will be used to select frames from each source. Next, declare a [ManualResetEventSlim](#), a [CancellationTokenSource](#), and an [EventHandler](#) that will be used to implement timeout logic for the example.

```

private MultiSourceMediaFrameReader _multiFrameReader = null;
private string _colorSourceId = null;
private string _depthSourceId = null;

private readonly ManualResetEventSlim _frameReceived = new
ManualResetEventSlim(false);
private readonly CancellationTokenSource _tokenSource = new
CancellationTokenSource();
public event EventHandler CorrelationFailed;

```

Using the techniques described previously in this article, query for a **MediaFrameSourceGroup** that includes the color and depth sources required for this example scenario. After selecting the desired frame source group, get the **MediaFrameSourceInfo** for each frame source.

C#

```

var allGroups = await MediaFrameSourceGroup.FindAllAsync();
var eligibleGroups = allGroups.Select(g => new
{
    Group = g,

    // For each source kind, find the source which offers that kind of media
    frame,
    // or null if there is no such source.
    SourceInfos = new MediaFrameSourceInfo[]
    {
        g.SourceInfos.FirstOrDefault(info => info.SourceKind ==
MediaFrameSourceKind.Color),
        g.SourceInfos.FirstOrDefault(info => info.SourceKind ==
MediaFrameSourceKind.Depth)
    }
}).Where(g => g.SourceInfos.Any(info => info != null)).ToList();

if (eligibleGroups.Count == 0)
{
    System.Diagnostics.Debug.WriteLine("No source group with color, depth or
infrared found.");
    return;
}

var selectedGroupIndex = 0; // Select the first eligible group
MediaFrameSourceGroup selectedGroup =
eligibleGroups[selectedGroupIndex].Group;
MediaFrameSourceInfo colorSourceInfo =
eligibleGroups[selectedGroupIndex].SourceInfos[0];
MediaFrameSourceInfo depthSourceInfo =
eligibleGroups[selectedGroupIndex].SourceInfos[1];

```

Create and initialize a **MediaCapture** object, passing the selected frame source group in the initialization settings.

C#

```
mediaCapture = new MediaCapture();

var settings = new MediaCaptureInitializationSettings()
{
    SourceGroup = selectedGroup,
    SharingMode = MediaCaptureSharingMode.ExclusiveControl,
    MemoryPreference = MediaCaptureMemoryPreference.Cpu,
    StreamingCaptureMode = StreamingCaptureMode.Video
};

await mediaCapture.InitializeAsync(settings);
```

After initializing the **MediaCapture** object, retrieve **MediaFrameSource** objects for the color and depth cameras. Store the ID for each source so that you can select the arriving frame for the corresponding source.

C#

```
MediaFrameSource colorSource =
    mediaCapture.FrameSources.Values.FirstOrDefault(
        s => s.Info.SourceKind == MediaFrameSourceKind.Color);

MediaFrameSource depthSource =
    mediaCapture.FrameSources.Values.FirstOrDefault(
        s => s.Info.SourceKind == MediaFrameSourceKind.Depth);

if (colorSource == null || depthSource == null)
{
    System.Diagnostics.Debug.WriteLine("MediaCapture doesn't have the Color
    and Depth streams");
    return;
}

_colorSourceId = colorSource.Info.Id;
_depthSourceId = depthSource.Info.Id;
```

Create and initialize the **MultiSourceMediaFrameReader** by calling **CreateMultiSourceFrameReaderAsync** and passing an array of frame sources that the reader will use. Register an event handler for the **FrameArrived** event. This example creates an instance the **FrameRenderer** helper class, described previously in this article, to render frames to an **Image** control. Start the frame reader by calling **StartAsync**.

Register an event handler for the **CorellationFailed** event declared earlier in the example. We will signal this event if one of the media frame sources being used stops producing frames. Finally, call **Task.Run** to call the timeout helper method, **NotifyAboutCorrelationFailure**, on a separate thread. The implementation of this method is shown later in this article.

C#

```
_multiFrameReader = await mediaCapture.CreateMultiSourceFrameReaderAsync(
    new[] { colorSource, depthSource });

_multiFrameReader.FrameArrived += MultiFrameReader_FrameArrived;

_frameRenderer = new FrameRenderer(imageElement);

MultiSourceMediaFrameReaderStartStatus startStatus =
    await _multiFrameReader.StartAsync();

if (startStatus != MultiSourceMediaFrameReaderStartStatus.Success)
{
    throw new InvalidOperationException(
        "Unable to start reader: " + startStatus);
}

this.CorrelationFailed += MainPage_CorrelationFailed;
Task.Run(() => NotifyAboutCorrelationFailure(_tokenSource.Token));
```

The **FrameArrived** event is raised whenever a new frame is available from all of the media frame sources that are managed by the **MultiSourceMediaFrameReader**. This means that the event will be raised on the cadence of the slowest media source. If one source produces multiple frames in the time that a slower source produces one frame, the extra frames from the fast source will be dropped.

Get the **MultiSourceMediaFrameReference** associated with the event by calling **TryAcquireLatestFrame**. Get the **MediaFrameReference** associated with each media frame source by calling **TryGetFrameReferenceBySourceId**, passing in the ID strings stored when the frame reader was initialized.

Call the **Set** method of the **ManualResetEventSlim** object to signal that frames have arrived. We will check this event in the **NotifyCorrelationFailure** method that is running in a separate thread.

Finally, perform any processing on the time-correlated media frames. This example simply displays the frame from the depth source.

C#

```

private void MultiFrameReader_FrameArrived(MultiSourceMediaFrameReader
sender, MultiSourceMediaFrameArrivedEventArgs args)
{
    using (MultiSourceMediaFrameReference muxedFrame =
        sender.TryAcquireLatestFrame())
    using (MediaFrameReference colorFrame =
        muxedFrame.TryGetFrameReferenceBySourceId(_colorSourceId))
    using (MediaFrameReference depthFrame =
        muxedFrame.TryGetFrameReferenceBySourceId(_depthSourceId))
    {
        // Notify the listener thread that the frame has been received.
        _frameReceived.Set();
        _frameRenderer.ProcessFrame(depthFrame);
    }
}

```

The **NotifyCorrelationFailure** helper method was run on a separate thread after the frame reader was started. In this method, check to see if the frame received event has been signaled. Remember, in the **FrameArrived** handler, we set this event whenever a set of correlated frames arrive. If the event hasn't been signaled for some app-defined period of time - 5 seconds is a reasonable value - and the task wasn't cancelled using the **CancellationToken**, then it's likely that one of the media frame sources has stopped reading frames. In this case you typically want to shut down the frame reader, so raise the app-defined **CorrelationFailed** event. In the handler for this event you can stop the frame reader and clean up it's associated resources as shown previously in this article.

C#

```

private void NotifyAboutCorrelationFailure(CancellationToken token)
{
    // If in 5 seconds the token is not cancelled and frame event is not
    // signaled,
    // correlation is most likely failed.
    if (WaitHandle.WaitAny(new[] { token.WaitHandle,
        _frameReceived.WaitHandle }, 5000)
        == WaitHandle.WaitTimeout)
    {
        CorrelationFailed?.Invoke(this, EventArgs.Empty);
    }
}

```

C#

```

private async void MainPage_CorrelationFailed(object sender, EventArgs e)
{
    await _multiFrameReader.StopAsync();
    _multiFrameReader.FrameArrived -= MultiFrameReader_FrameArrived;
    mediaCapture.Dispose();
}

```

```
mediaCapture = null;  
}
```

Use buffered frame acquisition mode to preserve the sequence of acquired frames

Starting with Windows 10, version 1709, you can set the [AcquisitionMode](#) property of a **MediaFrameReader** or **MultiSourceMediaFrameReader** to **Buffered** to preserve the sequence of frames passed into your app from the frame source.

C#

```
mediaFrameReader.AcquisitionMode = MediaFrameReaderAcquisitionMode.Buffered;
```

In the default acquisition mode, **Realtime**, if multiple frames are acquired from the source while your app is still handling the **FrameArrived** event for a previous frame, the system will send your app the most recently acquired frame and drop additional frames waiting in the buffer. This provides your app with the most recent available frame at all times. This is typically the most useful mode for realtime computer vision applications.

In **Buffered** acquisition mode, the system will keep all frames in the buffer and provide them to your app through the **FrameArrived** event in the order received. Note that in this mode, when system's buffer for frames is filled, the system will stop acquiring new frames until your app completes the **FrameArrived** event for previous frames, freeing up more space in the buffer.

Use MediaSource to display frames in a MediaPlayerElement

Starting with Windows, version 1709, you can display frames acquired from a **MediaFrameReader** directly in a **MediaPlayerElement** control in your XAML page. This is achieved by using the [MediaSource.CreateFromMediaFrameSource](#) to create **MediaSource** object that can be used directly by a **MediaPlayer** associated with a **MediaPlayerElement**. For detailed information on working with **MediaPlayer** and **MediaPlayerElement**, see [Play audio and video with MediaPlayer](#).

The following code examples show you a simple implementation that displays the frames from a front-facing and back-facing camera simultaneously in a XAML page.

First, add two **MediaPlayerElement** controls to your XAML page.

XML

```
<MediaPlayerElement x:Name="mediaPlayerElement1" Width="320" Height="240"/>
```

XML

```
<MediaPlayerElement x:Name="mediaPlayerElement2" Width="320" Height="240"/>
```

Next, using the techniques shown in previous sections in this article, select a **MediaFrameSourceGroup** that contains **MediaFrameSourceInfo** objects for color cameras on the front panel and back panel. Note that the **MediaPlayer** does not automatically convert frames from non-color formats, such as a depth or infrared data, into color data. Using other sensor types may produce unexpected results.

C#

```
var allGroups = await MediaFrameSourceGroup.FindAllAsync();
var eligibleGroups = allGroups.Select(g => new
{
    Group = g,

    // For each source kind, find the source which offers that kind of media
    frame,
    // or null if there is no such source.
    SourceInfos = new MediaFrameSourceInfo[]
    {
        g.SourceInfos.FirstOrDefault(info =>
            info.DeviceInformation?.EnclosureLocation.Panel ==
            Windows.Devices.Enumeration.Panel.Front
            && info.SourceKind == MediaFrameSourceKind.Color),
        g.SourceInfos.FirstOrDefault(info =>
            info.DeviceInformation?.EnclosureLocation.Panel ==
            Windows.Devices.Enumeration.Panel.Back
            && info.SourceKind == MediaFrameSourceKind.Color)
    }
}).Where(g => g.SourceInfos.Any(info => info != null)).ToList();

if (eligibleGroups.Count == 0)
{
    System.Diagnostics.Debug.WriteLine("No source group with front and back-
    facing camera found.");
    return;
}

var selectedGroupIndex = 0; // Select the first eligible group
MediaFrameSourceGroup selectedGroup =
    eligibleGroups[selectedGroupIndex].Group;
MediaFrameSourceInfo frontSourceInfo = selectedGroup.SourceInfos[0];
MediaFrameSourceInfo backSourceInfo = selectedGroup.SourceInfos[1];
```

Initialize the **MediaCapture** object to use the selected **MediaFrameSourceGroup**.

C#

```
mediaCapture = new MediaCapture();

var settings = new MediaCaptureInitializationSettings()
{
    SourceGroup = selectedGroup,
    SharingMode = MediaCaptureSharingMode.ExclusiveControl,
    MemoryPreference = MediaCaptureMemoryPreference.Cpu,
    StreamingCaptureMode = StreamingCaptureMode.Video
};
try
{
    await mediaCapture.InitializeAsync(settings);
}
catch (Exception ex)
{
    System.Diagnostics.Debug.WriteLine("MediaCapture initialization failed:
" + ex.Message);
    return;
}
```

Finally, call **MediaSource.CreateFromMediaFrameSource** to create a **MediaSource** for each frame source by using the **Id** property of the associated **MediaFrameSourceInfo** object to select one of the frame sources in the **MediaCapture** object's **FrameSources** collection. Initialize a new **MediaPlayer** object and assign it to a **MediaPlayerElement** by calling **SetMediaPlayer**. Then set the **Source** property to the newly created **MediaSource** object.

C#

```
var frameMediaSource1 =
MediaSource.CreateFromMediaFrameSource(mediaCapture.FrameSources[frontSource
Info.Id]);
mediaPlayerElement1.SetMediaPlayer(new
Windows.Media.Playback.MediaPlayer());
mediaPlayerElement1.MediaPlayer.Source = frameMediaSource1;
mediaPlayerElement1.AutoPlay = true;

var frameMediaSource2 =
MediaSource.CreateFromMediaFrameSource(mediaCapture.FrameSources[backSourceI
nfo.Id]);
mediaPlayerElement2.SetMediaPlayer(new
Windows.Media.Playback.MediaPlayer());
mediaPlayerElement2.MediaPlayer.Source = frameMediaSource2;
mediaPlayerElement2.AutoPlay = true;
```

Use video profiles to select a frame source

A camera profile, represented by a [MediaCaptureVideoProfile](#) object, represents a set of capabilities that a particular capture device provides, such as frame rates, resolutions, or advanced features like HDR capture. A capture device may support multiple profiles, allowing you to select the one that is optimized for your capture scenario. Starting with Windows 10, version 1803, you can use [MediaCaptureVideoProfile](#) to select a media frame source with particular capabilities before initializing the [MediaCapture](#) object. The following example method looks for a video profile that supports HDR with Wide Color Gamut (WCG) and returns a [MediaCaptureInitializationSettings](#) object that can be used to initialize the [MediaCapture](#) to use the selected device and profile.

First, call [MediaFrameSourceGroup.FindAllAsync](#) to get a list of all media frame source groups available on the current device. Loop through each source group and call [MediaCapture.FindKnownVideoProfiles](#) to get a list of all of the video profiles for the current source group that support the specified profile, in this case HDR with WCG photo. If a profile that meets the criteria is found, create a new [MediaCaptureInitializationSettings](#) object and set the [VideoProfile](#) to the select profile and the [VideoDeviceId](#) to the [Id](#) property of the current media frame source group.

C#

```
public async Task<MediaCaptureInitializationSettings>
FindHdrWithWcgPhotoProfile()
{
    IReadOnlyList<MediaFrameSourceGroup> sourceGroups = await
MediaFrameSourceGroup.FindAllAsync();
    MediaCaptureInitializationSettings settings = null;

    foreach (MediaFrameSourceGroup sourceGroup in sourceGroups)
    {
        // Find a device that support AdvancedColorPhoto
        IReadOnlyList<MediaCaptureVideoProfile> profileList =
MediaCapture.FindKnownVideoProfiles(
                                sourceGroup.Id,
                                KnownVideoProfile.HdrWithWcgPhoto);

        if (profileList.Count > 0)
        {
            settings = new MediaCaptureInitializationSettings();
            settings.VideoProfile = profileList[0];
            settings.VideoDeviceId = sourceGroup.Id;
            break;
        }
    }
    return settings;
}
```

```
private void StartDeviceWatcherButton_Click(object sender, RoutedEventArgs e)
{
    var remoteCameraHelper = new RemoteCameraPairingHelper(this.Dispatcher);
}
```

For more information on using camera profiles, see [Camera profiles](#).

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [Camera frames sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Process audio frames with MediaFrameReader

Article • 10/20/2022

This article shows you how to use a [MediaFrameReader](#) with [MediaCapture](#) to get audio data from a media frame source. To learn about using a **MediaFrameReader** to get image data, such as from a color, infrared, or depth camera, see [Process media frames with MediaFrameReader](#). That article provides a general overview of the frame reader usage pattern and discusses some additional features of the **MediaFrameReader** class, such as using **MediaFrameSourceGroup** to retrieve frames from multiple sources at the same time.

ⓘ Note

The features discussed in this article are only available starting with Windows 10, version 1803.

ⓘ Note

There is a Universal Windows app sample that demonstrates using **MediaFrameReader** to display frames from different frame sources, including color, depth, and infrared cameras. For more information, see [Camera frames sample](#) [↗].

Setting up your project

The process for acquiring audio frames is largely the same as acquiring other types of media frames. As with any app that uses **MediaCapture**, you must declare that your app uses the *webcam* capability before attempting to access any camera device. If your app will capture from an audio device, you should also declare the *microphone* device capability.

Add capabilities to the app manifest

1. In Microsoft Visual Studio, in **Solution Explorer**, open the designer for the application manifest by double-clicking the **package.appxmanifest** item.
2. Select the **Capabilities** tab.
3. Check the box for **Webcam** and the box for **Microphone**.

4. For access to the Pictures and Videos library check the boxes for **Pictures Library** and the box for **Videos Library**.

Select frame sources and frame source groups

The first step in capturing audio frames is to initialize a [MediaFrameSource](#) representing the source of the audio data, such as a microphone or other audio capture device. To do this, you must create a new instance of the [MediaCapture](#) object. For this example, the only initialization setting for the [MediaCapture](#) is setting the [StreamingCaptureMode](#) to indicate that we want to stream audio from the capture device.

After calling [MediaCapture.InitializeAsync](#), you can get the list of accessible media frame sources with the [FrameSources](#) property. This example uses a Linq query to select all frame sources where the [MediaFrameSourceInfo](#) describing the frame source has a [MediaStreamType](#) of **Audio**, indicating that the media source produces audio data.

If the query returns one or more frame sources, you can check the [CurrentFormat](#) property to see if the source supports the audio format you desire - in this example, float audio data. Check the [AudioEncodingProperties](#) to make sure the audio encoding you desire is supported by the source.

C#

```
mediaCapture = new MediaCapture();
MediaCaptureInitializationSettings settings = new
MediaCaptureInitializationSettings()
{
    StreamingCaptureMode = StreamingCaptureMode.Audio,
};
await mediaCapture.InitializeAsync(settings);

var audioFrameSources = mediaCapture.FrameSources.Where(x =>
x.Value.Info.MediaStreamType == MediaStreamType.Audio);

if (audioFrameSources.Count() == 0)
{
    Debug.WriteLine("No audio frame source was found.");
    return;
}

MediaFrameSource frameSource = audioFrameSources.FirstOrDefault().Value;

MediaFrameFormat format = frameSource.CurrentFormat;
if (format.Subtype != MediaEncodingSubtypes.Float)
{
    return;
}
```

```
if (format.AudioEncodingProperties.ChannelCount != 1
    || format.AudioEncodingProperties.SampleRate != 48000)
{
    return;
}
```

Create and Start the MediaFrameReader

Get a new instance of **MediaFrameReader** by calling

MediaCapture.CreateFrameReaderAsync, passing the **MediaFrameSource** object you selected in the previous step. By default, audio frames are obtained in buffered mode, making it less likely that frames will be dropped, although this can still occur if you are not processing audio frames fast enough and fill up the system's allotted memory buffer.

Register a handler for the **MediaFrameReader.FrameArrived** event, which is raised by the system when a new frame of audio data is available. Call **StartAsync** to begin the acquisition of audio frames. If the frame reader fails to start, the status value returned from the call will have a value other than **Success**.

C#

```
mediaFrameReader = await mediaCapture.CreateFrameReaderAsync(frameSource);

// Optionally set acquisition mode. Buffered is the default mode for audio.
mediaFrameReader.AcquisitionMode = MediaFrameReaderAcquisitionMode.Buffered;

mediaFrameReader.FrameArrived += MediaFrameReader_AudioFrameArrived;

var status = await mediaFrameReader.StartAsync();

if (status != MediaFrameReaderStartStatus.Success)
{
    Debug.WriteLine("The MediaFrameReader couldn't start.");
}
```

In the **FrameArrived** event handler, call **TryAcquireLatestFrame** on the **MediaFrameReader** object passed as the sender to the handler to attempt to retrieve a reference to the latest media frame. Note that this object can be null, so you should always check before using the object. The type of media frame wrapped in the **MediaFrameReference** returned from **TryAcquireLatestFrame** depends on what type of frame source or sources you configured the frame reader to acquire. Since the frame reader in this example was set up to acquire audio frames, it gets the underlying frame using the **AudioMediaFrame** property.

This **ProcessAudioFrame** helper method in the example below shows how to get an **AudioFrame** which provides information such as the timestamp of the frame and whether it is discontinuous from the **AudioMediaFrame** object. To read or process the audio sample data, you will need to get the **AudioBuffer** object from the **AudioMediaFrame** object, create an **IMemoryBufferReference**, and then call the COM method **IMemoryBufferByteAccess::GetBuffer** to retrieve the data. See the note below the code listing for more information on accessing native buffers.

The format of the data depends on the frame source. In this example, when selecting a media frame source, we explicitly made certain that the selected frame source used a single channel of float data. The rest of the example code shows how to determine the duration and sample count for the audio data in the frame.

C#

```
private void MediaFrameReader_AudioFrameArrived(MediaFrameReader sender,
MediaFrameArrivedEventArgs args)
{
    using (MediaFrameReference reference = sender.TryAcquireLatestFrame())
    {
        if (reference != null)
        {
            ProcessAudioFrame(reference.AudioMediaFrame);
        }
    }
}

unsafe private void ProcessAudioFrame(AudioMediaFrame audioMediaFrame)
{
    using (AudioFrame audioFrame = audioMediaFrame.GetAudioFrame())
    using (AudioBuffer buffer =
audioFrame.LockBuffer(AudioBufferAccessMode.Read))
    using (IMemoryBufferReference reference = buffer.CreateReference())
    {
        byte* dataInBytes;
        uint capacityInBytes;
        float* dataInFloat;

        ((IMemoryBufferByteAccess)reference).GetBuffer(out dataInBytes, out
capacityInBytes);

        // The requested format was float
        dataInFloat = (float*)dataInBytes;

        // Get the number of samples by multiplying the duration by sampling
rate:
        // duration [s] x sampling rate [samples/s] = # samples

        // Duration can be gotten off the frame reference OR the audioFrame
        TimeSpan duration = audioMediaFrame.FrameReference.Duration;
```

```

        // frameDurMs is in milliseconds, while SampleRate is given per
        second.
        uint frameDurMs = (uint)duration.TotalMilliseconds;
        uint sampleRate =
audioMediaFrame.AudioEncodingProperties.SampleRate;
        uint sampleCount = (frameDurMs * sampleRate) / 1000;

    }
}

```

⚠ Note

In order to do operate on the audio data, you must access a native memory buffer. To do this, you must use the **IMemoryBufferByteAccess** COM interface by including the code listing below. Operations on the native buffer must be performed in a method that uses the **unsafe** keyword. You also need to check the box to allow unsafe code in the **Build** tab of the **Project -> Properties** dialog.

C#

```

[ComImport]
[Guid("5B0D3235-4DBA-4D44-865E-8F1D0E4FD04D")]
[InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
unsafe interface IMemoryBufferByteAccess
{
    void GetBuffer(out byte* buffer, out uint capacity);
}

```

Additional information on using MediaFrameReader with audio data

You can retrieve the [AudioDeviceController](#) associated with the audio frame source by accessing the [MediaFrameSource.Controller](#) property. This object can be used to get or set the stream properties of the capture device or to control the capture level. The following example mutes the audio device so that frames continue to be acquired by the frame reader, but all samples have value of 0.

C#

```

audioDeviceController.Muted = true;

```

You can use an [AudioFrame](#) object to pass audio data captured by a media frame source into an [AudioGraph](#). Pass the frame into the [AddFrame](#) method of an [AudioFrameInputNode](#). For more information on using audio graphs to capture, process, and mix audio signals, see [Audio graphs](#).

Related topics

- [Process media frames with MediaFrameReader](#)
 - [Camera](#)
 - [Basic photo, video, and audio capture with MediaCapture](#)
 - [Camera frames sample](#) 
 - [Audio graphs](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Use the Open Source Computer Vision Library (OpenCV) with MediaFrameReader

Article • 10/20/2022

This article shows you how to use the Open Source Computer Vision Library (OpenCV), a native code library that provides a wide variety of image processing algorithms, with the [MediaFrameReader](#) class that can read media frames from multiple sources simultaneously. The example code in this article walks you through creating a simple app that obtains frames from a color sensor, blurs each frame using the OpenCV library, and then displays the processed image in a XAML **Image** control.

ⓘ Note

OpenCV.Win.Core and OpenCV.Win.ImgProc are not regularly updated and do not pass the Store compliance checks, therefore these packages are intended for experimentation only.

This article builds on the content of two other articles:

- [Process media frames with MediaFrameReader](#) - This article provides detailed information on using **MediaFrameReader** to obtain frames from one or more media frame sources and describes, in detail, most of the sample code in this article. In particular, **Process media frames with MediaFrameReader** provides the code listing for a helper class, **FrameRenderer**, which handles the presentation of media frames in a XAML **Image** element. The sample code in this article uses this helper class as well.
- [Process software bitmaps with OpenCV](#) - This article walks you through creating a native code Windows Runtime component, **OpenCVBridge**, which helps to convert between the **SoftwareBitmap** object, used by the **MediaFrameReader**, and the **Mat** type used by the OpenCV library. The sample code in this article assumes you have followed the steps to add the **OpenCVBridge** component to your UWP app solution.

In addition to these articles, to view and download a full, end-to-end working sample of the scenario described in this article, see the [Camera Frames + OpenCV Sample](#) in the Windows Universal Samples GitHub repo.

To get started developing quickly, you can include the OpenCV library in a UWP app project by using NuGet packages, but these packages may not pass the app certification process when you submit your app to the Store, so it is recommended that you download the OpenCV library source code and build the binaries yourself before submitting your app. Information developing with OpenCV can be found at <https://opencv.org>

Implement the OpenCVHelper native Windows Runtime component

Follow the steps in [Process software bitmaps with OpenCV](#) to create the OpenCV helper Windows Runtime component and add a reference to the component project to your UWP app solution.

Find available frame source groups

First, you need to find a media frame source group from which media frames will be obtained. Get the list of available source groups on the current device by calling [MediaFrameSourceGroup.FindAllAsync](#). Then select the source groups that provide the sensor types required for your app scenario. For this example, we simply need a source group that provides frames from an RGB camera.

C#

```
var frameSourceGroups = await MediaFrameSourceGroup.FindAllAsync();
var selectedGroupObjects = frameSourceGroups.Select(group =>
    new
    {
        sourceGroup = group,
        colorSourceInfo = group.SourceInfos.FirstOrDefault((sourceInfo) =>
            // On Xbox/Kinect, omit the MediaStreamType and EnclosureLocation
            tests
            return sourceInfo.SourceKind == MediaFrameSourceKind.Color;
        })
    })
    .Where(t => t.colorSourceInfo != null)
    .FirstOrDefault();

MediaFrameSourceGroup selectedGroup = selectedGroupObjects?.sourceGroup;
MediaFrameSourceInfo colorSourceInfo =
    selectedGroupObjects?.colorSourceInfo;

if (selectedGroup == null)
{
```

```
    return;  
}
```

Initialize the MediaCapture object

Next, you need to initialize the **MediaCapture** object to use the frame source group selected in the previous step by setting the **SourceGroup** property of the **MediaCaptureInitializationSettings**.

ⓘ Note

The technique used by the **OpenCVHelper** component, described in detail in [Process software bitmaps with OpenCV](#), requires that the image data resides in CPU memory, not GPU memory. So, you should specify **MemoryPreference.CPU** for the **MemoryPreference** field of the **MediaCaptureInitializationSettings**.

After the **MediaCapture** object has been initialized, get a reference to the RGB frame source by accessing the **MediaCapture.FrameSources** property.

C#

```
mediaCapture = new MediaCapture();  
  
var settings = new MediaCaptureInitializationSettings()  
{  
    SourceGroup = selectedGroup,  
    SharingMode = MediaCaptureSharingMode.ExclusiveControl,  
    MemoryPreference = MediaCaptureMemoryPreference.Cpu,  
    StreamingCaptureMode = StreamingCaptureMode.Video  
};  
try  
{  
    await mediaCapture.InitializeAsync(settings);  
}  
catch (Exception ex)  
{  
    System.Diagnostics.Debug.WriteLine("MediaCapture initialization failed:  
" + ex.Message);  
    return;  
}  
  
var colorFrameSource = mediaCapture.FrameSources[colorSourceInfo.Id];
```

Initialize the MediaFrameReader

Next, create a [MediaFrameReader](#) for the RGB frame source retrieved in the previous step. In order to maintain a good frame rate, you may want to process frames that have a lower resolution than the resolution of the sensor. This example provides the optional [BitmapSize](#) argument to the [MediaCapture.CreateFrameReaderAsync](#) method to request that frames provided by the frame reader be resized to 640 x 480 pixels.

After creating the frame reader, register a handler for the [FrameArrived](#) event. Then create a new [SoftwareBitmapSource](#) object, which the [FrameRenderer](#) helper class will use to present the processed image. Then call the constructor for the [FrameRenderer](#). Initialize the instance of the [OpenCVHelper](#) class defined in the [OpenCVBridge](#) Windows Runtime component. This helper class is used in the [FrameArrived](#) handler to process each frame. Finally, start the frame reader by calling [StartAsync](#).

C#

```
BitmapSize size = new BitmapSize() // Choose a lower resolution to make the
image processing more performant
{
    Height = 480,
    Width = 640
};

mediaFrameReader = await
mediaCapture.CreateFrameReaderAsync(colorFrameSource,
MediaEncodingSubtypes.Argb32, size);
mediaFrameReader.FrameArrived += ColorFrameReader_FrameArrived_OpenCV;

imageElement.Source = new SoftwareBitmapSource();
_frameRenderer = new FrameRenderer(imageElement);

await mediaFrameReader.StartAsync();
```

Handle the FrameArrived event

The [FrameArrived](#) event is raised whenever a new frame is available from the frame reader. Call [TryAcquireLatestFrame](#) to get the frame, if it exists. Get the [SoftwareBitmap](#) from the [MediaFrameReference](#). Note that the [CVHelper](#) class used in this example requires images to use the BRGA8 pixel format with premultiplied alpha. If the frame passed into the event has a different format, convert the [SoftwareBitmap](#) to the correct format. Next, create a [SoftwareBitmap](#) to be used as the target of the blur operation. The source image properties are used as arguments to the constructor to create a bitmap with matching format. Call the helper class [Blur](#) method to process the frame. Finally, pass the output image of the blur operation into [PresentSoftwareBitmap](#), the

method of the **FrameRenderer** helper class that displays the image in the XAML **Image** control with which it was initialized.

C#

```
private void ColorFrameReader_FrameArrived_OpenCV(MediaFrameReader sender,
MediaFrameArrivedEventArgs args)
{
    var mediaFrameReference = sender.TryAcquireLatestFrame();
    if (mediaFrameReference != null)
    {
        SoftwareBitmap openCVInputBitmap = null;
        var inputBitmap =
mediaFrameReference.VideoMediaFrame?.SoftwareBitmap;
        if (inputBitmap != null)
        {
            //The XAML Image control can only display images in BGRA8 format
            with premultiplied or no alpha
            if (inputBitmap.BitmapPixelFormat == BitmapPixelFormat.Bgra8
                && inputBitmap.BitmapAlphaMode ==
BitmapAlphaMode.Premultiplied)
            {
                openCVInputBitmap = SoftwareBitmap.Copy(inputBitmap);
            }
            else
            {
                openCVInputBitmap = SoftwareBitmap.Convert(inputBitmap,
BitmapPixelFormat.Bgra8, BitmapAlphaMode.Premultiplied);
            }

            SoftwareBitmap openCVOutputBitmap = new
SoftwareBitmap(BitmapPixelFormat.Bgra8, openCVInputBitmap.PixelWidth,
openCVInputBitmap.PixelHeight, BitmapAlphaMode.Premultiplied);

            // operate on the image and render it
            openCVHelper.Blur(openCVInputBitmap, openCVOutputBitmap);
            _frameRenderer.PresentSoftwareBitmap(openCVOutputBitmap);
        }
    }
}
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [Process media frames with MediaFrameReader](#)
- [Process software bitmaps with OpenCV](#)

- [Camera frames sample](#) ↗
- [Camera Frames + OpenCV Sample](#) ↗

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) ↗ | [Get help at Microsoft Q&A](#)

Capture from multiple sources using MediaFrameSourceGroup

Article • 10/20/2022

This article shows you how to capture video from multiple sources simultaneously to a single file with multiple embedded video tracks. Starting with RS3, you can specify multiple [VideoStreamDescriptor](#) objects for a single [MediaEncodingProfile](#). This enables you to encode multiple streams simultaneously to a single file. The video streams that are encoded in this operation must be included in a single [MediaFrameSourceGroup](#) which specifies a set of cameras on the current device that can be used at the same time.

For information on using [MediaFrameSourceGroup](#) with the [MediaFrameReader](#) class to enable real-time computer vision scenarios that use multiple cameras, see [Process media frames with MediaFrameReader](#).

The rest of this article will walk you through the steps of recording video from two color cameras to a single file with multiple video tracks.

Find available sensor groups

A [MediaFrameSourceGroup](#) represents a collection of frame sources, typically cameras, that can be accessed simultaneously. The set of available frame source groups is different for each device, so the first step in this example is to get the list of available frame source groups and finding one that contains the necessary cameras for the scenario, which in this case requires two color cameras.

The [MediaFrameSourceGroup.FindAllAsync](#) method returns all source groups available on the current device. Each returned [MediaFrameSourceGroup](#) has a list of [MediaFrameSourceInfo](#) objects that describes each frame source in the group. A Linq query is used to find a source group that contains two color cameras, one on the front panel and one on the back. An anonymous object is returned that contains the selected [MediaFrameSourceGroup](#) and the [MediaFrameSourceInfo](#) for each color camera. Instead of using Linq syntax, you could instead loop through each group, and then each [MediaFrameSourceInfo](#) to look for a group that meets your requirements.

Note that not every device will contain a source group that contains two color cameras, so you should check to make sure that a source group was found before trying to capture video.

C#

```
var sensorGroups = await MediaFrameSourceGroup.FindAllAsync();

var foundGroup = sensorGroups.Select(g => new
{
    group = g,
    color1 = g.SourceInfos.Where(info => info.SourceKind ==
MediaFrameSourceKind.Color && info.DeviceInformation.EnclosureLocation.Panel
== Windows.Devices.Enumeration.Panel.Front).FirstOrDefault(),
    color2 = g.SourceInfos.Where(info => info.SourceKind ==
MediaFrameSourceKind.Color && info.DeviceInformation.EnclosureLocation.Panel
== Windows.Devices.Enumeration.Panel.Back).FirstOrDefault()
}).Where(g => g.color1 != null && g.color2 != null).FirstOrDefault();

if (foundGroup == null)
{
    Debug.WriteLine("No groups found.");
    return;
}
```

Initialize the MediaCapture object

The [MediaCapture](#) class is the primary class that is used for most audio, video, and photo capture operations in UWP apps. Initialize the object by calling [InitializeAsync](#), passing in a [MediaCaptureInitializationSettings](#) object that contains initialization parameters. In this example, the only specified setting is the [SourceGroup](#) property, which is set to the **MediaFrameSourceGroup** that was retrieved in the previous code example.

For information on other operations you can perform with **MediaCapture** and other UWP app features for capturing media, see [Camera](#).

C#

```
var settings = new MediaCaptureInitializationSettings()
{
    SourceGroup = foundGroup.group
};

mediaCapture = new MediaCapture();
await mediaCapture.InitializeAsync(settings);
```

Create a MediaEncodingProfile

The [MediaEncodingProfile](#) class tells the media capture pipeline how captured audio and video should be encoded as they are written to a file. For typical capture and transcoding scenarios, this class provides a set of static methods for creating common profiles, like [CreateAvi](#) and [CreateMp3](#). For this example, an encoding profile is manually created using an Mpeg4 container and H264 video encoding. Video encoding settings are specified using a [VideoEncodingProperties](#) object. For each color camera used in this scenario, a [VideoStreamDescriptor](#) object is configured. The descriptor is constructed with the [VideoEncodingProperties](#) object specifying the encoding. The [Label](#) property of the [VideoStreamDescriptor](#) must be set to the ID of the media frame source that will be captured to the stream. This is how the capture pipeline knows which stream descriptor and encoding properties should be used for each camera. The ID of the frame source is exposed by the [MediaFrameSourceInfo](#) objects that were found in the previous section, when a [MediaFrameSourceGroup](#) was selected.

Starting with Windows 10, version 1709, you can set multiple encoding properties on a [MediaEncodingProfile](#) by calling [SetVideoTracks](#). You can retrieve the list of video stream descriptors by calling [GetVideoTracks](#). Note that if you set the [Video](#) property, which stores a single stream descriptor, the descriptor list you set by calling [SetVideoTracks](#) will be replaced with a list containing the single descriptor you specified.

C#

```
var profile = new MediaEncodingProfile();
profile.Container = new ContainerEncodingProperties();
profile.Container.Subtype = MediaEncodingSubtypes.Mpeg4;

List<VideoStreamDescriptor> streams = new List<VideoStreamDescriptor>();

var encodeProps = VideoEncodingProperties.CreateH264();
encodeProps.Subtype = MediaEncodingSubtypes.H264;
var stream1Desc = new VideoStreamDescriptor(encodeProps);
stream1Desc.Label = foundGroup.color1.Id;
streams.Add(stream1Desc);

var encodeProps2 = VideoEncodingProperties.CreateH264();
encodeProps2.Subtype = MediaEncodingSubtypes.H264;
var stream2Desc = new VideoStreamDescriptor(encodeProps2);
stream2Desc.Label = foundGroup.color2.Id;
streams.Add(stream2Desc);

profile.SetVideoTracks(streams);
profile.Audio = null;
```

Encode timed metadata in media files

Starting with Windows 10, version 1803, in addition to audio and video you can encode timed metadata into a media file for which the data format is supported. For example, GoPro metadata (gpmc) can be stored in MP4 files to convey the geographic location correlated with a video stream.

Encoding metadata uses a pattern that is parallel to encoding audio or video. The [TimedMetadataEncodingProperties](#) class describes the type, subtype and encoding properties of the metadata, like [VideoEncodingProperties](#) does for video. The [TimedMetadataStreamDescriptor](#) identifies a metadata stream, just as the [VideoStreamDescriptor](#) does for video streams.

The following example shows how to initialize a [TimedMetadataStreamDescriptor](#) object. First, a [TimedMetadataEncodingProperties](#) object is created and the **Subtype** is set to a GUID that identifies the type of metadata that will be included in the stream. This example uses the GUID for GoPro metadata (gpmc). The [SetFormatUserData](#) method is called to set format-specific data. For MP4 files, the format-specific data is stored in the SampleDescription box (stsd). Next, a new [TimedMetadataStreamDescriptor](#) is created from the encoding properties. The **Label** and **Name** properties are set to identify the stream to be encoded.

C#

```
        TimedMetadataEncodingProperties encodingProperties = new
TimedMetadataEncodingProperties
    {
        Subtype = "{67706D64-BF10-48B4-BC18-593DC1DB950F}"
    };

    byte[] streamDescriptionData =
GetStreamDescriptionDataForGpmcEncodingSubtype();
    encodingProperties.SetFormatUserData(streamDescriptionData);

    TimedMetadataStreamDescriptor descriptor = new
TimedMetadataStreamDescriptor(encodingProperties)
    {
        Name = "GPS Info",
        Label = "GPS Info"
    };
};
```

Call [MediaEncodingProfile.SetTimedMetadataTracks](#) to add the metadata stream descriptor to the encoding profile. The following example shows a helper method that takes two video stream descriptors, one audio stream descriptor, and one timed metadata stream descriptor and returns a [MediaEncodingProfile](#) that can be used to encode the streams.

C#

```

public MediaEncodingProfile CreateProfileForTranscoder(VideoStreamDescriptor
videoStream1, VideoStreamDescriptor videoStream2, AudioStreamDescriptor
audioStream, TimedMetadataStreamDescriptor timedMetadataStream)
{
    ContainerEncodingProperties container = new
ContainerEncodingProperties()
    {
        Subtype = MediaEncodingSubtypes.Mpeg4
    };

    MediaEncodingProfile profile = new MediaEncodingProfile()
    {
        Container = container
    };

    VideoStreamDescriptor encodingVideoStream1 = videoStream1.Copy();
    encodingVideoStream1.EncodingProperties.Subtype =
MediaEncodingSubtypes.H264;
    encodingVideoStream1.Label = videoStream1.Name;

    VideoStreamDescriptor encodingVideoStream2 = videoStream2.Copy();
    encodingVideoStream2.EncodingProperties.Subtype =
MediaEncodingSubtypes.H264;
    encodingVideoStream2.Label = videoStream2.Name;

    AudioStreamDescriptor encodingAudioStream = audioStream.Copy();
    encodingAudioStream.EncodingProperties.Subtype =
MediaEncodingSubtypes.Ac3;
    encodingAudioStream.Label = audioStream.Name;

    TimedMetadataStreamDescriptor encodingTimedMetadataStream =
timedMetadataStream.Copy();

    profile.SetTimedMetadataTracks(new TimedMetadataStreamDescriptor[] {
encodingTimedMetadataStream });
    profile.SetVideoTracks(new VideoStreamDescriptor[] {
encodingVideoStream1, encodingVideoStream2 });
    profile.SetAudioTracks(new AudioStreamDescriptor[] { encodingAudioStream
});
    return profile;
}

```

Record using the multi-stream MediaEncodingProfile

The final step in this example is to initiate video capture by calling

[StartRecordToStorageFileAsync](#), passing in the **StorageFile** to which the captured media

is written, and the **MediaEncodingProfile** created in the previous code example. After waiting a few seconds, the recording is stopped with a call to [StopRecordAsync](#).

C#

```
var recordFile = await
Windows.Storage.KnownFolders.CameraRoll.CreateFileAsync("record.mp4",
Windows.Storage.CreationCollisionOption.GenerateUniqueName);
await mediaCapture.StartRecordToStorageFileAsync(profile, recordFile);
await Task.Delay(8000);
await mediaCapture.StopRecordAsync();
```

When the operation is complete, a video file will have been created that contains the video captured from each camera encoded as a separate stream within the file. For information on playing media files containing multiple video tracks, see [Media items, playlists, and tracks](#).

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [Process media frames with MediaFrameReader](#)
- [Media items, playlists, and tracks](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Connect to remote cameras

Article • 10/20/2022

This article shows you how to connect to one or more remote cameras and get a [MediaFrameSourceGroup](#) object that allows you to read frames from each camera. For more information on reading frames from a media source, see [Process media frames with MediaFrameReader](#). For more information on pairing with devices, see [Pair devices](#).

ⓘ Note

The features discussed in this article are available starting with Windows 10, version 1903.

Create a DeviceWatcher class to watch for available remote cameras

The [DeviceWatcher](#) class monitors the devices available to your app and notifies your app when devices are added or removed. Get an instance of **DeviceWatcher** by calling [DeviceInformation.CreateWatcher](#), passing in an Advanced Query Syntax (AQS) string that identifies the type of devices you want to monitor. The AQS string specifying network camera devices is the following:

syntax

```
@"System.Devices.InterfaceClassGuid:=""{B8238652-B500-41EB-B4F3-4234F7F5AE99}"" AND  
System.Devices.InterfaceEnabled:=System.StructuredQueryType.Boolean#True"
```

ⓘ Note

The helper method [MediaFrameSourceGroup.GetDeviceSelector](#) returns an AQS string that will monitor locally-connected and remote network cameras. To monitor only network cameras, you should use the AQS string shown above.

When you start the returned **DeviceWatcher** by calling the [Start](#) method, it will raise the [Added](#) event for every network camera that is currently available. Until you stop the watcher by calling [Stop](#), the **Added** event will be raised when new network camera

devices become available and the **Removed** event will be raised when a camera device becomes unavailable.

The event args passed into the **Added** and **Removed** event handlers are a **DeviceInformation** or a **DeviceInformationUpdate** object, respectively. Each of these objects has an **Id** property that is the identifier for the network camera for which the event was fired. Pass this ID into the **MediaFrameSourceGroup.FromIdAsync** method to get a **MediaFrameSourceGroup** object that you can use to retrieve frames from the camera.

Remote camera pairing helper class

The following example shows a helper class that uses a **DeviceWatcher** to create and update an **ObservableCollection** of **MediaFrameSourceGroup** objects to support data binding to the list of cameras. Typical apps would wrap the **MediaFrameSourceGroup** in a custom model class. Note that the helper class maintains a reference to the app's **CoreDispatcher** and updates the collection of cameras within calls to **RunAsync** to ensure that the UI bound to the collection is updated on the UI thread.

Also, this example handles the **DeviceWatcher.Updated** event in addition to the **Added** and **Removed** events. In the **Updated** handler, the associated remote camera device is removed from and then added back to the collection.

C#

```
class RemoteCameraPairingHelper : IDisposable
{
    private CoreDispatcher _dispatcher;
    private DeviceWatcher _watcher;
    private ObservableCollection<MediaFrameSourceGroup>
_remoteCameraCollection;
    public RemoteCameraPairingHelper(CoreDispatcher uiDispatcher)
    {
        _dispatcher = uiDispatcher;
        _remoteCameraCollection = new
ObservableCollection<MediaFrameSourceGroup>();
        var remoteCameraAqs = @"System.Devices.InterfaceClassGuid:=""
{B8238652-B500-41EB-B4F3-4234F7F5AE99}"" AND
System.Devices.InterfaceEnabled:=System.StructuredQueryType.Boolean#True";
        _watcher = DeviceInformation.CreateWatcher(remoteCameraAqs);
        _watcher.Added += Watcher_Added;
        _watcher.Removed += Watcher_Removed;
        _watcher.Updated += Watcher_Updated;
        _watcher.Start();
    }
    public void Dispose()
    {

```

```

        _watcher.Stop();
        _watcher.Updated -= Watcher_Updated;
        _watcher.Removed -= Watcher_Removed;
        _watcher.Added -= Watcher_Added;
    }
    public IReadOnlyList<MediaFrameSourceGroup> FrameSourceGroups
    {
        get { return _remoteCameraCollection; }
    }
    private async void Watcher_Updated(DeviceWatcher sender,
DeviceInformationUpdate args)
    {
        await RemoveDevice(args.Id);
        await AddDeviceAsync(args.Id);
    }
    private async void Watcher_Removed(DeviceWatcher sender,
DeviceInformationUpdate args)
    {
        await RemoveDevice(args.Id);
    }
    private async void Watcher_Added(DeviceWatcher sender, DeviceInformation
args)
    {
        await AddDeviceAsync(args.Id);
    }
    private async Task AddDeviceAsync(string id)
    {
        var group = await MediaFrameSourceGroup.FromIdAsync(id);
        if (group != null)
        {
            await _dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
            {
                _remoteCameraCollection.Add(group);
            });
        }
    }
    private async Task RemoveDevice(string id)
    {
        await _dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
        {
            var existing = _remoteCameraCollection.FirstOrDefault(item =>
item.Id == id);
            if (existing != null)
            {
                _remoteCameraCollection.Remove(existing);
            }
        });
    }
}

```

C++/WinRT

```

#include <winrt/Windows.Devices.Enumeration.h>
#include <winrt/Windows.Foundation.Collections.h>

```

```

#include <winrt/Windows.Media.Capture.Frames.h>
#include <winrt/Windows.UI.Core.h>
using namespace winrt;
using namespace winrt::Windows::Devices::Enumeration;
using namespace winrt::Windows::Foundation::Collections;
using namespace winrt::Windows::Media::Capture::Frames;
using namespace winrt::Windows::UI::Core;

struct RemoteCameraPairingHelper
{
    RemoteCameraPairingHelper(CoreDispatcher uiDispatcher) :
        m_dispatcher(uiDispatcher)
    {
        m_remoteCameraCollection =
winrt::single_threaded_observable_vector<MediaFrameSourceGroup>();
        auto remoteCameraAqs =
            LR"(System.Devices.InterfaceClassGuid:=\"{B8238652-B500-41EB-
B4F3-4234F7F5AE99}\"")
            LR"(AND
System.Devices.InterfaceEnabled:=System.StructuredQueryType.Boolean#True)";
        m_watcher = DeviceInformation::CreateWatcher(remoteCameraAqs);
        m_watcherAddedAutoRevoker = m_watcher.Added(winrt::auto_revoke, {
this, &RemoteCameraPairingHelper::Watcher_Added });
        m_watcherRemovedAutoRevoker = m_watcher.Removed(winrt::auto_revoke,
{ this, &RemoteCameraPairingHelper::Watcher_Removed });
        m_watcherUpdatedAutoRevoker = m_watcher.Updated(winrt::auto_revoke,
{ this, &RemoteCameraPairingHelper::Watcher_Updated });
        m_watcher.Start();
    }
    ~RemoteCameraPairingHelper()
    {
        m_watcher.Stop();
    }
    IObservableVector<MediaFrameSourceGroup> FrameSourceGroups()
    {
        return m_remoteCameraCollection;
    }
    winrt::fire_and_forget Watcher_Added(DeviceWatcher /* sender */,
DeviceInformation args)
    {
        co_await AddDeviceAsync(args.Id());
    }
    winrt::fire_and_forget Watcher_Removed(DeviceWatcher /* sender */,
DeviceInformationUpdate args)
    {
        co_await RemoveDevice(args.Id());
    }
    winrt::fire_and_forget Watcher_Updated(DeviceWatcher /* sender */,
DeviceInformationUpdate args)
    {
        co_await RemoveDevice(args.Id());
        co_await AddDeviceAsync(args.Id());
    }
    Windows::Foundation::IAsyncAction AddDeviceAsync(winrt::hstring id)
    {

```

```

        auto group = co_await MediaFrameSourceGroup::FromIdAsync(id);
        if (group)
        {
            co_await m_dispatcher;
            m_remoteCameraCollection.Append(group);
        }
    }
Windows::Foundation::IAsyncAction RemoveDevice(winrt::hstring id)
{
    co_await m_dispatcher;

    uint32_t ix{ 0 };
    for (auto const&& item : m_remoteCameraCollection)
    {
        if (item.Id() == id)
        {
            m_remoteCameraCollection.RemoveAt(ix);
            break;
        }
        ++ix;
    }
}

private:
    CoreDispatcher m_dispatcher{ nullptr };
    DeviceWatcher m_watcher{ nullptr };
    IObservableVector<MediaFrameSourceGroup> m_remoteCameraCollection;
    DeviceWatcher::Added_revoker m_watcherAddedAutoRevoker;
    DeviceWatcher::Removed_revoker m_watcherRemovedAutoRevoker;
    DeviceWatcher::Updated_revoker m_watcherUpdatedAutoRevoker;
};

```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [Camera frames sample](#) 
- [Process media frames with MediaFrameReader](#)

Get a preview frame

Article • 07/29/2024

This topic shows you how to get a single preview frame from the media capture preview stream.

ⓘ Note

This article builds on concepts and code discussed in [Basic photo, video, and audio capture with MediaCapture](#), which describes the steps for implementing basic photo and video capture. We recommend that you familiarize yourself with the basic media capture pattern in that article before moving on to more advanced capture scenarios. The code in this article assumes that your app already has an instance of `MediaCapture` that has been properly initialized, and that you have a [CaptureElement](#) with an active video preview stream.

In addition to the namespaces required for basic media capture, capturing a preview frame requires the following namespace.

C#

```
using Windows.Media;
```

When you request a preview frame, you can specify the format in which you would like to receive the frame by creating a `VideoFrame` object with the format you desire. This example creates a video frame that is the same resolution as the preview stream by calling `VideoDeviceController.GetMediaStreamProperties` and specifying `MediaStreamType.VideoPreview` to request the properties for the preview stream. The width and height of the preview stream is used to create the new video frame.

C#

```
// Get information about the preview
var previewProperties =
_mediaCapture.VideoDeviceController.GetMediaStreamProperties(MediaStreamType
.VideoPreview) as VideoEncodingProperties;

// Create a video frame in the desired format for the preview frame
VideoFrame videoFrame = new VideoFrame(BitmapPixelFormat.Bgra8,
(int)previewProperties.Width, (int)previewProperties.Height);
```

If your [MediaCapture](#) object is initialized and you have an active preview stream, call [GetPreviewFrameAsync](#) to get a preview stream. Pass in the video frame created in the last step to specify the format of the returned frame.

C#

```
VideoFrame previewFrame = await  
_mediaCapture.GetPreviewFrameAsync(videoFrame);
```

Get a [SoftwareBitmap](#) representation of the preview frame by accessing the [SoftwareBitmap](#) property of the [VideoFrame](#) object. For information about saving, loading, and modifying software bitmaps, see [Imaging](#).

C#

```
SoftwareBitmap previewBitmap = previewFrame.SoftwareBitmap;
```

You can also get an [IDirect3DSurface](#) representation of the preview frame if you want to use the image with Direct3D APIs.

C#

```
var previewSurface = previewFrame.Direct3DSurface;
```

Important

Either the [SoftwareBitmap](#) property or the [Direct3DSurface](#) property of the returned [VideoFrame](#) may be null depending on how you call [GetPreviewFrameAsync](#) and also depending on the device on which your app is running.

- If you call the overload of [GetPreviewFrameAsync](#) that accepts a [VideoFrame](#) argument, the returned [VideoFrame](#) will have a non-null [SoftwareBitmap](#) and the [Direct3DSurface](#) property will be null.
- If you call the overload of [GetPreviewFrameAsync](#) that has no arguments on a device that uses a Direct3D surface to represent the frame internally, the [Direct3DSurface](#) property will be non-null and the [SoftwareBitmap](#) property will be null.
- If you call the overload of [GetPreviewFrameAsync](#) that has no arguments on a device that does not use a Direct3D surface to represent the frame internally,

the **SoftwareBitmap** property will be non-null and the **Direct3DSurface** property will be null.

Your app should always check for a null value before trying to operate on the objects returned by the **SoftwareBitmap** or **Direct3DSurface** properties.

When you are done using the preview frame, be sure to call its **Close** method (projected to **Dispose** in C#) to free the resources used by the frame. Or, use the **using** pattern, which automatically disposes of the object.

C#

```
previewFrame.Dispose();  
previewFrame = null;
```

Related topics

- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Launch the camera settings page

Article • 11/26/2024

Windows defines a set of URIs that allow apps to launch the Windows Settings app and display a particular settings page. This article explains how to launch the Windows Settings app directly to the camera settings page and, optionally, navigate directly to the settings for a particular camera on the device. For more information, see [Launch the Windows Settings app](#).

The camera settings URL

Starting with Windows 11, Build 22000, the URI `ms-settings:camera` launches the Windows Settings app and navigates to the camera settings page. Note that in previous versions of Windows, this same URI would launch the default camera application. In addition to the general camera settings page, you can append the query string parameter `cameraId` set to the symbolic link name, in escaped URI format, to launch directly to the settings page for the associated camera.

In the following example, the [DeviceInformation](#) class is used to retrieve the symbolic link name for the first video capture device on the current machine, if one exists. Next, [LaunchUriAsync](#) is called to launch the Windows Settings app. The `ms-settings:camera` Uri specifies that the camera settings page should be shown. The optional query string parameter `cameraId` is set to the symbolic link name for the camera, escaped with a call to [Url.EscapeDataString](#), to specify that the settings for the associated camera should be shown.

C#

```
private async void LaunchSettingsPage_Click(object sender, RoutedEventArgs e)
{
    var captureDevices = await
DeviceInformation.FindAllAsync(DeviceClass.VideoCapture);
    if (captureDevices.Count() > 0)
    {
        var cameraSymbolicLink = captureDevices.First().Id;
        bool result = await Windows.System.Launcher.LaunchUriAsync(new
Uri("ms-settings:camera?cameraId=" +
Uri.EscapeDataString(cameraSymbolicLink)));
    }
}
```